

Case Study Assignment - Inventory Management System for B2B SaaS

Github Repo - https://github.com/MukundC25/case_study

Part 1: Code Review & Debugging (30 minutes)

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Your Tasks:

1. **Identify Issues:** List all problems you see with this code (technical and business logic)
2. **Explain Impact:** For each issue, explain what could go wrong in production
3. **Provide Fixes:** Write the corrected version with explanations

Additional Context (you may need to ask for more):

- Products can exist in multiple warehouses
- SKUs must be unique across the platform
- Price can be decimal values
- Some fields might be optional

– Issues Identified

Issue 1 – request.json can return None

Problem: `request.json` (deprecated alias for `get_json()`) returns `None` when the Content-Type header is not `application/json` or the body is malformed.

The very next line `data['name']` then raises `TypeError: NoneType object is not subscriptable`.

Production impact: Every caller that forgets the correct Content-Type header gets a raw 500.

Fix: Use `request.get_json(silent=True)` and return a 400 immediately when data is `None`.

Issue 2 – No input validation / missing-field protection

Problem: Any missing key in data raises an unhandled `KeyError`.

Production impact: A client omitting any one of `name`, `sku`, `price`, `warehouse_id`, or `initial_quantity` causes a 500 with a Python stack trace exposed to the caller.

Fix: Explicitly check for required fields before touching the DB.

Issue 3 – Two separate `db.session.commit()` calls (non-atomic operation)

Problem: The Product is committed first; the Inventory creation happens in a second transaction. If anything fails between the two commits (DB timeout, process restart, invalid `initial_quantity` type, etc.) the product row is persisted permanently but has no corresponding inventory record.

Production impact: Data inconsistency — orphaned products with no inventory. Queries that join products to inventory will silently drop these products, and attempts to re-create them will fail with a duplicate-SKU error.

Fix: Use `db.session.flush()` after adding the product (to get the generated `product.id` without committing), then add the inventory, and issue one `db.session.commit()` at the end. Wrap both in `try/except` and call `db.session.rollback()` on any failure.

Issue 4 – No exception handling

Problem: There is no `try/except` block anywhere in the function.

Production impact: Any runtime error (DB connectivity issue, constraint violation, type error) returns a 500 with a full stack trace — a security and UX problem.

Fix: Wrap the entire DB section in `try/except IntegrityError` (for constraint violations) and a broad `except Exception` fallback that returns a clean JSON error.

Issue 5 – No price validation

Problem: `data['price']` is passed directly to the Product constructor with no type check.

Production impact: A string value like "free" or a negative number like -5 would be stored without complaint (depending on the DB column type), corrupting billing calculations.

Fix: Parse price via `Decimal(str(data['price']))` and reject values < 0 .

Issue 6 – No initial_quantity validation

Problem: `initial_quantity` could be a float, a string, or a negative integer.

Production impact: Negative stock corrupts reports; a float silently truncates in some DB drivers; a non-numeric string causes an unhandled exception.

Fix: Cast to `int()` and reject values < 0 .

Issue 7 – No warehouse existence check

Problem: `data['warehouse_id']` is used directly without verifying the warehouse exists.

Production impact: A foreign-key violation is thrown at commit time, surfacing as a raw 500 instead of a descriptive 404 / 400.

Fix: Query the warehouse first; return 404 if not found.

Issue 8 – No SKU uniqueness check + race condition

Problem: There is no application-level SKU uniqueness check. If the DB has a unique constraint on `sku`, an `IntegrityError` is raised and surfaces as a 500. If it doesn't have a constraint, duplicate SKUs are silently stored.

Even with an explicit check (if `Product.query.filter_by(sku=sku).first()`), two concurrent requests with the same SKU can both pass the check before either commits — a classic TOCTOU race condition.

Production impact: Duplicate SKUs break inventory lookups; clients get no actionable error message.

Fix:

1. Add a `UNIQUE` constraint on `products.sku` at the DB level (the true guard).
2. Add an application-level pre-check to return a friendly 409 Conflict.
3. Catch `IntegrityError` in the except block as a second line of defence against races.

Issue 9 – Wrong HTTP status code (200 instead of 201)

Problem: The function returns a plain dict with Flask's default 200 OK.

Production impact: Clients following REST conventions expect 201 Created for successful resource creation. Returns 200 also prevents caching and API gateway rules from treating it as a creation event.

Fix: Return `jsonify({...}), 201`.

Issue 10 – Product.warehouse_id is an architectural design flaw

Problem: Storing warehouse_id directly on Product means each product can only belong to one warehouse. The stated requirement is "products can exist in multiple warehouses."

Production impact: Can only track stock in one location per product; adding a product to a second warehouse would require duplicating the product row with a different SKU or misusing the schema.

Fix: Remove warehouse_id from Product. The warehouse relationship lives exclusively in the Inventory junction table (product_id, warehouse_id, quantity), which supports the many-to-many requirement natively.

– Fixed Implementation

See app/routes/products.py in the repository. Key changes summarised:

```
@products_bp.route("/api/products", methods=["POST"])
def create_product():
    data = request.get_json(silent=True)

    if not data:
        return jsonify({"error": "..."}), 400

    # Validate all required fields

    # Validate price as Decimal, quantity as non-negative int

    # Verify warehouse exists → 404 if not

    # Pre-check SKU uniqueness → 409 if duplicate

    try:
        product = Product(name=name, sku=sku, price=price, ...)
        db.session.add(product)

        db.session.flush()      # get product.id without committing

        inventory = Inventory(product_id=product.id, ...)

        db.session.add(inventory)

        db.session.commit()     # single atomic commit
```

```
except IntegrityError:
```

```
    db.session.rollback()
```

```
    return jsonify({"error": "SKU already exists"}), 409
```

```
except Exception:
```

```
    db.session.rollback()
```

```
    return jsonify({"error": "Unexpected error"}), 500
```

```
return jsonify({"message": "Product created", "product_id": product.id}), 201
```

Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note:** These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

Your Tasks:

1. **Design Schema:** Create tables with columns, data types, and relationships
2. **Identify Gaps:** List questions you'd ask the product team about missing requirements
3. **Explain Decisions:** Justify your design choices (indexes, constraints, etc.)

Format: Use any notation (SQL DDL, ERD, text description, etc.)

– Schema (full DDL in `schema.sql`)

```
companies
```

```
|— id, name, created_at, updated_at
```

warehouses

└─ id, company_id → companies.id

└─ name, address, is_active, created_at

product_types

└─ id, name (UNIQUE)

└─ default_low_stock_threshold ← drives alert thresholds per category

products

└─ id, company_id → companies.id

└─ product_type_id → product_types.id (nullable)

└─ name, sku (UNIQUE), price (NUMERIC 10,2 ≥ 0)

└─ description, is_bundle, is_active, created_at, updated_at

inventory ← product × warehouse junction

└─ id

└─ product_id → products.id

└─ warehouse_id → warehouses.id

└─ quantity (≥ 0)

└─ low_stock_threshold (nullable) ← per-location override

└─ UNIQUE(product_id, warehouse_id)

inventory_transactions ← immutable audit log

└─ id, inventory_id → inventory.id

└─ quantity_change (+ add, - remove)

└─ transaction_type ENUM(restock|sale|adjustment|transfer_in|transfer_out)

└─ reference_id (order_id / transfer_id etc.)

└─ notes, created_at, created_by

suppliers

└─ id, company_id → companies.id

└─ name, contact_email, contact_phone, address, is_active

product_suppliers ← many-to-many product ↔ supplier

└─ id, product_id, supplier_id

└─ unit_cost, lead_time_days

└─ is_primary ← UNIQUE partial index ensures one primary per product

└─ UNIQUE(product_id, supplier_id)

bundle_items ← defines bundle composition

└─ id, bundle_product_id, component_product_id, quantity

└─ CHECK bundle_product_id ≠ component_product_id

└─ UNIQUE(bundle_product_id, component_product_id)

– Design Decisions & Justifications

Decision	Rationale
inventory is a junction table, not a column on products	Products can exist in multiple warehouses; quantity is a property of the (product, warehouse) pair, not the product itself

inventory_transactions is an append-only audit log	Enables reliable sales-velocity calculations, full stock history, and audit compliance; quantity on inventory is the materialized current state
Three-tier threshold resolution	Product-type default covers 80% of cases; per-location override handles edge cases (e.g. seasonal safety stock at one warehouse) without needing a schema change
is_primary flag on product_suppliers with a partial unique index	Enforces exactly one primary supplier per product at the DB level, preventing dual-primary bugs in application code
price NUMERIC(10,2)	Avoids floating-point rounding errors for financial values
is_active soft-delete flags	Preserves transaction history and referential integrity; hard deletes would orphan audit records
CHECK (quantity >= 0) on inventory	Database-level guard; prevents negative stock from ever being stored, even if application code has a bug
Indexes on FK columns and (transaction_type, created_at)	The alert query filters transactions by type and date range — a composite index on these two columns makes it a fast index scan instead of a full table scan

– Identified Gaps / Questions for the Product Team

1. SKU scope – Are SKUs unique per company, or is the sku column truly global across all companies? (Current design: global unique.)
2. Bundle inventory tracking – When a bundle is sold, do we decrement the bundle's inventory, the components' inventory, or both?
3. Deleted warehouses – What happens to inventory records when a warehouse is deactivated? Should stock be transferred or archived?
4. "Recent" window definition – What is the authoritative window for "recent sales activity"? (I defaulted to 30 days.)

5. Reserved inventory – Is there a concept of *reserved* stock (e.g. pending orders)? Available stock = quantity – reserved would change threshold logic.
 6. Multi-currency – Are all prices stored in a single currency, or do we need a `currency_code` column and FX conversion?
 7. User authentication – The `created_by` field on `inventory_transactions` references a `users` table not yet defined. Do users belong to companies?
 8. Cross-company products – Can a supplier's product exist in multiple companies' catalogs, or does each company maintain its own product list?
 9. Reorder quantities – Should `product_suppliers` also track a `minimum_order_quantity` for generating accurate reorder suggestions?
 10. Negative adjustments – The CHECK (quantity >= 0) constraint will block any adjustment transaction that would take stock negative. Is that the correct business rule, or should we allow "backorders"?
-

Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET `/api/companies/{company_id}/alerts/low-stock`

Expected Response Format:

```
{
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "supplier": {
        "id": 789,
        "name": "Supplier Corp",
```

```

        "contact_email": "orders@supplier.com"
    }
}
],
"total_alerts": 1
}

```

Your Tasks:

1. **Write Implementation:** Use any language/framework (Python/Flask, Node.js/Express, etc.)
2. **Handle Edge Cases:** Consider what could go wrong
3. **Explain Approach:** Add comments explaining your logic

Hints: You'll need to make assumptions about the database schema and business logic. Document these assumptions.

– Endpoint

GET /api/companies/{company_id}/alerts/low-stock

See app/routes/alerts.py for the full implementation.

– Algorithm

1. Validate company exists → 404 if not
2. Query inventory_transactions for SALE events in the last 30 days,
grouped by inventory_id → sales_map { inventory_id: total_sold }
3. Return early with empty alerts if no recent sales exist
4. Fetch all Inventory rows for the company's active warehouses
that appear in sales_map (i.e., had recent sales)
5. For each inventory row:
 - a. Resolve threshold (per-location → product_type → global default 10)
 - b. Skip if quantity >= threshold
 - c. Compute days_until_stockout = quantity / avg_daily_sales
(null if avg rounds to 0)

- d. Look up primary supplier
 - e. Append alert dict
6. Sort: soonest stockout first, then by lowest current stock
7. Return { alerts, total_alerts }

– Key Design Choices

- Two-query approach (sales subquery then inventory fetch): avoids a single massive JOIN that would pull the full transaction history per inventory row. The sales aggregation is done once in the DB; Python only touches rows that actually need alerting.
- Sorting by urgency: clients can display the most critical items at the top without any additional work.
- Null days_until_stockout: returned explicitly rather than omitted, so clients can distinguish "stockout imminent in 0 days" from "no velocity data available."

– Assumptions Documented

Assumption	Notes
"Recent sales" window = 30 days	Would confirm with PM; configurable via RECENT_SALES_WINDOW_DAYS constant
"Activity" = ≥ 1 sale in window	Non-moving slow stock excluded deliberately
days_until_stockout uses simple linear projection	A more sophisticated model would weight recent days more heavily
Bundle alerts treated same as regular products	Unclear if a bundle alert should cascade to components — needs clarification
Primary supplier only in response	If no primary set, supplier is null in the response

– Edge Cases Handled

Scenario	Behaviour
Company not found	404 { "error": "Company not found" }
Company has no warehouses	Returns { "alerts": [], "total_alerts": 0 }
No sales in the last 30 days	Returns { "alerts": [], "total_alerts": 0 }
Product below threshold but no supplier linked	Alert included; supplier field is null
avg_daily_sales rounds to 0	days_until_stockout: null
Inactive product or warehouse	Excluded from results

The screenshot shows a code editor with a Python script on the left and its JSON output in the terminal on the right.

Python Script (run.py):

```
1 import os
2 from app import create_app, db
3
4 app = create_app(os.environ.get("FLASK_ENV", "development"))
5
6 if __name__ == "__main__":
7     # ... (rest of the script) ...
```

Terminal Output:

```
{
  "alerts": [
    {
      "current_stock": 10,
      "days_until_stockout": 7,
      "product_id": 3,
      "product_name": "Steel Rod",
      "sku": "STR-001",
      "supplier": {
        "contact_email": "orders@supplier.com",
        "id": 1,
        "name": "Supplier Corp"
      },
      "threshold": 50,
      "warehouse_id": 1,
      "warehouse_name": "Main Warehouse"
    },
    {
      "current_stock": 5,
      "days_until_stockout": 10,
      "product_id": 1,
      "product_name": "Widget A",
      "sku": "WID-001",
      "supplier": {
        "contact_email": "orders@supplier.com",
        "id": 1,
        "name": "Supplier Corp"
      },
      "threshold": 20,
      "warehouse_id": 1,
      "warehouse_name": "Main Warehouse"
    },
    {
      "current_stock": 3,
      "days_until_stockout": 11,
      "product_id": 1,
      "product_name": "Widget A",
      "sku": "WID-001",
      "supplier": {
        "contact_email": "orders@supplier.com",
        "id": 1,
        "name": "Supplier Corp"
      },
      "threshold": 20,
      "warehouse_id": 2,
      "warehouse_name": "East Warehouse"
    },
    {
      "current_stock": 25,
      "days_until_stockout": 150,
      "product_id": 4,
      "product_name": "Packaging Box",
      "sku": "PKG-001",
      "supplier": {
        "contact_email": "orders@supplier.com",
        "id": 1,
        "name": "Supplier Corp"
      },
      "threshold": 30,
      "warehouse_id": 1,
      "warehouse_name": "Main Warehouse"
    }
  ],
  "total_alerts": 4
}
```

Follow the next steps to run the project. If not running the instance, then directly find the [stockflow.db](#) and csv tables in the GitHub repo 'Instance' folder.

– Running the Project

1. Install dependencies

```
pip install -r requirements.txt
```

2. Seed the database with demo data

```
python seed.py
```

3. Start the server

```
python run.py
```

4. Test the low-stock alerts endpoint

```
curl http://localhost:5001/api/companies/1/alerts/low-stock
```

5. Test the fixed create-product endpoint

```
curl -X POST http://localhost:5001/api/products \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"name":"Test  
Widget","sku":"TST-999","price":9.99,"warehouse_id":1,"initial_quantity":50}'
```

Total time: ~85 minutes
